```
# VENKATESH GAURI SHANKAR
```

```
##Exp 5: Outlier estimation and data cleaning using quartile and visualization methods.
```

```
import pandas as pd
import numpy as np
from functools import reduce
```

```
## 1. Dropping irrelevant columns
```

```
df = pd.read_csv('BL-Flickr-Images-Book.csv')
df.head()
```

	Identifier	Edition Statement	Place of Publication	Date of Publication	Publisher	Title	Author	Contributo
0	206	NaN	London	1879 [1878]	S. Tinsley & Co.	Walter Forbes. [A novel.] By A. A	A. A.	FORBE Walt
1	216	NaN	London; Virtue & Yorston	1868	Virtue & Co.	All for Greed. [A novel. The dedication signed...	A., A. A.	BLAZE D BURY, Ma Pauline Ros Barone
2	218	NaN	London	1869	Bradbury, Evans & Co.	Love the Avenger. By the author of “All for Gr...	A., A. A.	BLAZE D BURY, Ma Pauline Ros Barone
3	472	NaN	London	1851	James Darling	Welsh Sketches, chiefly ecclesiastical, to the...	A., E. S.	Appleya Erne Silvanu
4	480	A new edition, revised, etc.	London	1857	Wertheim & Macintosh	[The World in which I live, and my place in it...	A., E. S.	BROOM John Hen

```
to_drop = ['Edition Statement',
           'Corporate Author',
           'Corporate Contributors',
           'Former owner',
           'Engraver',
           'Contributors',
           'Issuance type',
           'Shelfmarks']

df.drop(to_drop, inplace = True, axis = 1)
df.head()
```

	Identifier	Place of Publication	Date of Publication	Publisher	Title	Author	
0	206	London	1879 [1878]	S. Tinsley & Co.	Walter Forbes. [A novel.] By A. A	A. A.	http://www
1	216	London; Virtue & Yorston	1868	Virtue & Co.	All for Greed. [A novel. The dedication signed...	A., A. A.	http://www
2	218	London	1869	Bradbury, Evans & Co.	Love the Avenger. By the author of “All for Gr...	A., A. A.	http://www
3	472	London	1851	James Darling	Welsh Sketches, chiefly ecclesiastical, to the...	A., E. S.	http://www

```
## 2.   Set the index of the dataset
```

```
df.set_index('Identifier', inplace = True)
df.head()
```

Identifier	Place of Publication	Date of Publication	Publisher	Title	Author	
206	London	1879 [1878]	S. Tinsley & Co.	Walter Forbes. [A novel.] By A. A	A. A.	http://www
216	London; Virtue & Yorston	1868	Virtue & Co.	All for Greed. [A novel. The dedication signed...	A., A. A.	http://www
218	London	1869	Bradbury, Evans & Co.	Love the Avenger. By the author of “All for Gr...	A., A. A.	http://www
472	London	1851	James Darling	Welsh Sketches, chiefly ecclesiastical, to the...	A., E. S.	http://www
480	London	1857	Wertheim & Macintosh	[The World in which I live, and my place in it...	A., E. S.	http://www

```
df['Date of Publication'].head(25)
```

```
Identifier
206            1879 [1878]
216                   1868
218                   1869
472                   1851
480                   1857
481                   1875
519                   1872
667                    NaN
874                   1676
1143                  1679
1280                  1802
1808                  1859
1905                  1888
1929           1839, 38-54
2836                  1897
2854                  1865
2956               1860-63
2957                  1873
3017                  1866
3131                  1899
```

```
4598                  1814
4884                  1820
4976                  1800
5382    1847, 48 [1846-48]
5385               [1897?]
Name: Date of Publication, dtype: object
```

```
## 3. Cleaning columns using the .apply function
```

```
unwanted_characters = ['[', ',', '-']

def clean_dates(item):
    dop= str(item.loc['Date of Publication'])

    if dop == 'nan' or dop[0] == '[':
        return np.NaN

    for character in unwanted_characters:
        if character in dop:
            character_index = dop.find(character)
            dop = dop[:character_index]

    return dop

df['Date of Publication'] = df.apply(clean_dates, axis = 1)
```

```
df.head()
```

Identifier	Place of Publication	Date of Publication	Publisher	Title	Author	
206	London	1879	S. Tinsley & Co.	Walter Forbes. [A novel.] By A. A	A. A.	http://www
216	London; Virtue & Yorston	1868	Virtue & Co.	All for Greed. [A novel. The dedication signed...	A., A. A.	http://www
218	London	1869	Bradbury, Evans & Co.	Love the Avenger. By the author of “All for Gr...	A., A. A.	http://www
472	London	1851	James Darling	Welsh Sketches, chiefly ecclesiastical, to the...	A., E. S.	http://www
480	London	1857	Wertheim & Macintosh	[The World in which I live, and my place in it...	A., E. S.	http://www

```
## 4. Using .str methods to clean columns
```

```
pub = df['Place of Publication']
df['Place of Publication'] = np.where(pub.str.contains('London'), 'London',
    np.where(pub.str.contains('Oxford'), 'Oxford',
        np.where(pub.eq('Newcastle upon Tyne'),'Newcastle-upon-Tyne', df['Place of Publication'])))
```

```
df.head()
```

Identifier	Place of Publication	Date of Publication	Publisher	Title	Author	
206	London	1879	S. Tinsley & Co.	Walter Forbes. [A novel.] By A. A	A. A.	http://www.flickr.com/photos/b
216	London	1868	Virtue & Co.	All for Greed. [A novel. The dedication signed...	A., A. A.	http://www.flickr.com/photos/b
218	London	1869	Bradbury, Evans &	Love the Avenger. By	A., A.	http://www.flickr.com/photos/b

5. Cleaning entire dataset

```
!more university_towns.txt
```

```
Alabama[edit]
Auburn (Auburn University)[1]
Florence (University of North Alabama)
Jacksonville (Jacksonville State University)[2]
Livingston (University of West Alabama)[2]
Montevallo (University of Montevallo)[2]
Troy (Troy University)[2]
Tuscaloosa (University of Alabama, Stillman College, Shelton State)[3][4]
Tuskegee (Tuskegee University)[5]
Alaska[edit]
Fairbanks (University of Alaska Fairbanks)[2]
Arizona[edit]
Flagstaff (Northern Arizona University)[6]
Tempe (Arizona State University)
Tucson (University of Arizona)
Arkansas[edit]
Arkadelphia (Henderson State University, Ouachita Baptist University)[2]
Conway (Central Baptist College, Hendrix College, University of Central Arkansas
)[2]
Fayetteville (University of Arkansas)[7]
Jonesboro (Arkansas State University)[8]
Magnolia (Southern Arkansas University)[2]
Monticello (University of Arkansas at Monticello)[2]
```

```
university_towns = []

with open('university_towns.txt', 'r') as file:
    items = file.readlines()
    states = list(filter(lambda x: '[edit]' in x, items))

    for index, state in enumerate(states):
        start = items.index(state) + 1
        if index == 49: #since 50 states
            end = len(items)
        else:
            end = items.index(states[index + 1])

        pairs = map(lambda x: [state, x], items[start:end])
        university_towns.extend(pairs)
```

```
towns_df = pd.DataFrame(university_towns, columns = ['State', 'RegionName'])
towns_df.head()
```

	State	RegionName
0	Alabama[edit]\n	Auburn (Auburn University)[1]\n
1	Alabama[edit]\n	Florence (University of North Alabama)\n
2	Alabama[edit]\n	Jacksonville (Jacksonville State University)[2]\n
3	Alabama[edit]\n	Livingston (University of West Alabama)[2]\n
4	Alabama[edit]\n	Montevallo (University of Montevallo)[2]\n

```
def clean_up(item):
    if '(' in item:
        return item[:item.find('(') - 1]

    if '[' in item:
        return item[:item.find('[')]


towns_df =  towns_df.applymap(clean_up)
towns_df.head()
```

	State	RegionName
0	Alabama	Auburn
1	Alabama	Florence
2	Alabama	Jacksonville
3	Alabama	Livingston
4	Alabama	Montevallo

```
##6.    Renaming columns and skipping rows
```

```
olympics_df = pd.read_csv('olympics.csv')
olympics_df.head()
```

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	NaN	? Summer	01 !	02 !	03 !	Total	? Winter	01 !	02 !	03 !	Total	? Games	01 !	02 !	03 !	Combined total
1	Afghanistan (AFG)	13	0	0	2	2	0	0	0	0	0	13	0	0	2	2
2	Algeria (ALG)	12	5	2	8	15	3	0	0	0	0	15	5	2	8	15

```
olympics_df = pd.read_csv('olympics.csv', skiprows = 1, header = 0)
olympics_df.head()
```

	Unnamed: 0	? Summer	01 !	02 !	03 !	Total	? Winter	01 !.1	02 !.1	03 !.1	Total.1	? Games	01 !.2	02 !.2	03 !.2
0	Afghanistan (AFG)	13	0	0	2	2	0	0	0	0	0	13	0	0	2
1	Algeria (ALG)	12	5	2	8	15	3	0	0	0	0	15	5	2	8
2	Argentina	23	18	24	28	70	18	0	0	0	0	41	18	24	28

```
new_names =  {'Unnamed: 0': 'Country',
              '? Summer': 'Summer Olympics',
              '01 !': 'Gold',
              '02 !': 'Silver',
              '03 !': 'Bronze',
              '? Winter': 'Winter Olympics',
              '01 !.1': 'Gold.1',
              '02 !.1': 'Silver.1',
              '03 !.1': 'Bronze.1',
              '? Games': '# Games',
              '01 !.2': 'Gold.2',
              '02 !.2': 'Silver.2',
              '03 !.2': 'Bronze.2'}

olympics_df.rename(columns = new_names, inplace = True)
```

```
olympics_df.head()
```

	Country	Summer Olympics	Gold	Silver	Bronze	Total	Winter Olympics	Gold.1	Silver.1	Bronze.1	Tot
0	Afghanistan (AFG)	13	0	0	2	2	0	0	0	0	
1	Algeria (ALG)	12	5	2	8	15	3	0	0	0	
2	Argentina (ARG)	23	18	24	28	70	18	0	0	0	
3	Armenia (ARM)	5	1	2	9	12	6	0	0	0	
	Australasia										

```
print(df.shape)
print(df.info())
```

```
(8287, 6)
<class 'pandas.core.frame.DataFrame'>
Int64Index: 8287 entries, 206 to 4160339
Data columns (total 6 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Place of Publication  8287 non-null   object
 1   Date of Publication   7320 non-null   object
 2   Publisher             4092 non-null   object
 3   Title                 8287 non-null   object
```

```
 4   Author               6509 non-null   object
 5   Flickr URL           8287 non-null   object
dtypes: object(6)
memory usage: 453.2+ KB
None
```

```
df.describe()
```

	Place of Publication	Date of Publication	Publisher	Title	Author	
count	8287	7320	4092	8287	6509	
unique	1192	403	1989	8210	5005	
				[A General Gazetteer,	Shakespeare,	

##7. Box -whisker plot and quartile

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Reading the data
df = pd.read_csv("data_out.csv")
print(df.shape)
print(df.info())
```

```
(500, 15)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 15 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Unnamed: 0         500 non-null    int64
 1   Loan_ID            500 non-null    object
 2   Gender             491 non-null    object
 3   Married            497 non-null    object
 4   Dependents         488 non-null    object
 5   Education          500 non-null    object
 6   Self_Employed      473 non-null    object
 7   ApplicantIncome    500 non-null    int64
 8   CoapplicantIncome  500 non-null    float64
 9   LoanAmount         482 non-null    float64
 10  Loan_Amount_Term   486 non-null    float64
 11  Credit_History     459 non-null    float64
 12  Property_Area      500 non-null    object
 13  Loan_Status        500 non-null    object
 14  Total_Income       500 non-null    object
dtypes: float64(4), int64(2), object(9)
memory usage: 58.7+ KB
None
```

```
df.describe()
```

	Unnamed: 0	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_His
count	500.000000	500.000000	500.000000	482.000000	486.000000	459.00
mean	249.500000	5493.644000	1506.307840	144.020747	342.543210	0.84
std	144.481833	6515.668972	2134.432188	82.344919	63.834977	0.36
min	0.000000	150.000000	0.000000	17.000000	12.000000	0.00
25%	124.750000	2874.500000	0.000000	100.000000	360.000000	1.00
50%	249.500000	3854.000000	1125.500000	126.500000	360.000000	1.00
75%	374.250000	5764.000000	2253.250000	161.500000	360.000000	1.00

```
#IQR
```

```
Q1 = df.quantile(0.25)
Q3 = df.quantile(0.75)
IQR = Q3 - Q1
print(IQR)
```

```
Unnamed: 0           249.50
ApplicantIncome     2889.50
CoapplicantIncome   2253.25
LoanAmount            61.50
Loan_Amount_Term       0.00
Credit_History         0.00
dtype: float64
```

```
#MILD
```

```
(print(df < (Q1 - 1.5 * IQR)) |(df > (Q3 + 1.5 * IQR)))
```

```
     ApplicantIncome  CoapplicantIncome  ...  Total_Income  Unnamed: 0
0              False              False  ...         False       False
1              False              False  ...         False       False
2              False              False  ...         False       False
3              False              False  ...         False       False
4              False              False  ...         False       False
..               ...                ...  ...           ...         ...
495            False              False  ...         False       False
496            False              False  ...         False       False
497            False              False  ...         False       False
498            False              False  ...         False       False
499            False              False  ...         False       False

[500 rows x 15 columns]
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/usr/local/lib/python3.7/dist-packages/pandas/core/ops/array_ops.py in na_logical_op(x, y, op)
    265         #  (xint or xbool) and (yint or bool)
--> 266         result = op(x, y)
    267     except TypeError:
```

```
#Extreme
```

```
TypeError: unsupported operand type(s) for |: 'NoneType' and 'bool'
```

```
(print(df < (Q1 - 3 * IQR)) |(df > (Q3 + 3 * IQR)))
```

```
     ApplicantIncome  CoapplicantIncome  ...  Total_Income  Unnamed: 0
0              False              False  ...         False       False
1              False              False  ...         False       False
2              False              False  ...         False       False
3              False              False  ...         False       False
4              False              False  ...         False       False
..               ...                ...  ...           ...         ...
495            False              False  ...         False       False
```

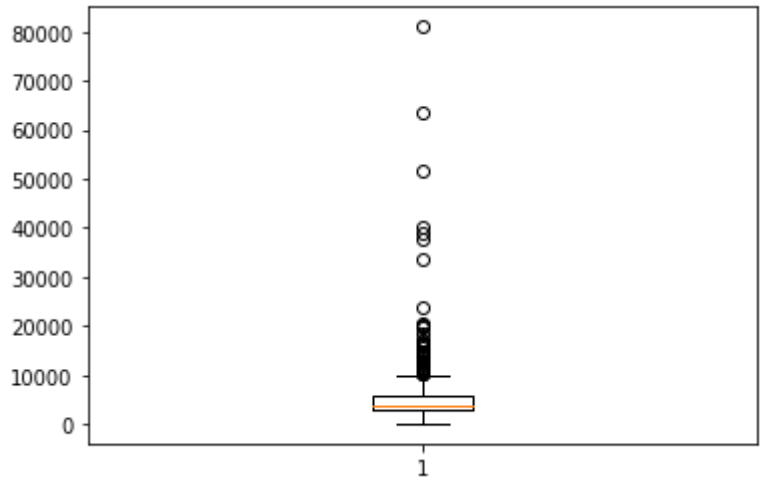
##Check any mild or extreme

```
498            False              False  ...         False       False
```

```
print(df['ApplicantIncome'].skew())
df['ApplicantIncome'].describe()
```

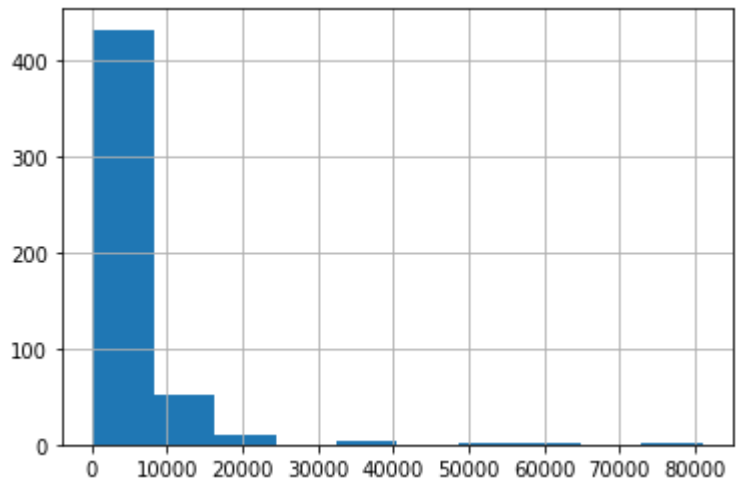
```
6.490441184579311
count      500.000000
mean      5493.644000
std       6515.668972
min        150.000000
25%       2874.500000
50%       3854.000000
75%       5764.000000
max      81000.000000
Name: ApplicantIncome, dtype: float64
```

```
plt.boxplot(df["ApplicantIncome"])
plt.show()
```




```
df.ApplicantIncome.hist()
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7f540e4f03d0>
```



```
#Quantile method
```

```
print(df['ApplicantIncome'].quantile(0.10))
print(df['ApplicantIncome'].quantile(0.90))
```

```
2213.9
9504.4
```

```
df["ApplicantIncome"] = np.where(df["ApplicantIncome"] <2213.9, 2213.9,df['ApplicantIncome'])
df["ApplicantIncome"] = np.where(df["ApplicantIncome"] >9504.0, 9504.0,df['ApplicantIncome'])
```

```
print(df['ApplicantIncome'].skew())
```

```
1.0086952477816755
```

```
#MILD
```

```
df_out = df[~((df < (Q1 - 1.5 * IQR)) |(df > (Q3 + 1.5 * IQR))).any(axis=1)]
print(df_out.shape)
```

```
(340, 15)
```

```
df_out
```

	Unnamed: 0	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncom
0	0	LP001002	Male	No	0	Graduate	No	5849.
1	1	LP001003	Male	Yes	1	Graduate	No	4583.
2	2	LP001005	Male	Yes	0	Graduate	Yes	3000.
3	3	LP001006	Male	Yes	0	Not Graduate	No	2583.
4	4	LP001008	Male	No	0	Graduate	No	6000.
...	...	...	...	...	...	...	...	.
491	491	LP002562	Male	Yes	1	Not Graduate	No	5333.
492	492	LP002571	Male	No	0	Not Graduate	No	3691.
493	493	LP002582	Female	No	0	Not Graduate	Yes	9504.
496	496	LP002587	Male	Yes	0	Not Graduate	No	2600.
498	498	LP002600	Male	Yes	1	Graduate	Yes	2895.

340 rows × 15 columns

```
#Extreme
```

```
df_out1 = df[~((df < (Q1 - 3 * IQR)) |(df > (Q3 + 3 * IQR))).any(axis=1)]
print(df_out1.shape)
```

```
(364, 15)
```

```
df_out1.count
```

```
<bound method DataFrame.count of      Unnamed: 0   Loan_ID  Gender  ... Property_Area
Loan_Status Total_Income
0             0  LP001002    Male  ...         Urban           Y      $5849.0
1             1  LP001003    Male  ...         Rural           N      $6091.0
2             2  LP001005    Male  ...         Urban           Y      $3000.0
3             3  LP001006    Male  ...         Urban           Y      $4941.0
4             4  LP001008    Male  ...         Urban           Y      $6000.0
..          ...       ...     ...  ...           ...         ...          ...
491         491  LP002562    Male  ...         Urban           Y      $6464.0
492         492  LP002571    Male  ...         Rural           Y      $3691.0
493         493  LP002582  Female  ...     Semiurban           Y     $17263.0
496         496  LP002587    Male  ...         Rural           Y      $4300.0
498         498  LP002600    Male  ...     Semiurban           Y      $2895.0

[364 rows x 15 columns]>
```

```
print(df_out['ApplicantIncome'].skew())
```

```
1.2061840472604732
```

```
print(df_out1['ApplicantIncome'].skew())
```

```
1.0854820239467449
```

```
##No of extreme outliers
```

```
df_out2= df[((df < (Q1 - 3 * IQR)) |(df > (Q3 + 3 * IQR))).any(axis=1)]
print(df_out2.shape)
```

```
(136, 15)
```

```
df_out2.count
```

```
<bound method DataFrame.count of      Unnamed: 0   Loan_ID  Gender  ... Property_Area
Loan_Status Total_Income
7             7  LP001014    Male  ...     Semiurban           N      $5540.0
9             9  LP001020    Male  ...     Semiurban           N     $23809.0
14           14  LP001030    Male  ...         Urban           Y      $2385.0
16           16  LP001034    Male  ...         Urban           Y      $3596.0
17           17  LP001036  Female  ...         Urban           N      $3510.0
..          ...       ...     ...  ...           ...         ...          ...
487         487  LP002547    Male  ...         Urban           N     $18333.0
494         494  LP002585    Male  ...         Rural           N      $5754.0
495         495  LP002586  Female  ...     Semiurban           Y      $4239.0
497         497  LP002588    Male  ...         Urban           Y      $7482.0
499         499  LP002602    Male  ...         Rural           N     $10699.0

[136 rows x 15 columns]>
```